



รายงาน PS02: Process Creation & Pipes

เสนอ

อาจารย์ ราชวิชช์ สโรชวิกสิต

จัดทำโดย

นายภูมิพัฒน์ อภิวัตนะพงศ์	67070501035
นายวิรัช ทองอุทัยศรี	67070501041
นายเจษฎา เกียรติกมลวงค์	67070501080

รายงานนี้เป็นส่วนหนึ่งของวิชา CPE 333 Operating Systems
ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์
ภาคการศึกษาที่ 1 ปีการศึกษา 2569
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

Contents

สภาพแวดล้อมที่ใช้ในการทดลอง

ระบบปฏิบัติการ	Ubuntu 24.04.1 LTS (บน WSL2 / Windows 11)
Kernel	6.6.87.2-microsoft-standard-WSL2 (ตรวจด้วย <code>uname -r</code>)
คอมไพเลอร์	gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
คำสั่งคอมไพล์	gcc -Wall -Wextra -o q1_1 q1_1_helloworld.c gcc -Wall -Wextra -o q1_2 q1_2_wait.c gcc -Wall -Wextra -o q2_1 q2_1_child_first.c gcc -Wall -Wextra -o q2_2 q2_2_parent_first.c gcc -Wall -Wextra -o q3_forkcount q3_forkcount.c
ผลการคอมไพล์	ผ่านทุกไฟล์ ไม่มี warning

หมายเหตุเรื่อง WSL2 WSL2 รัน Linux kernel จริง ดังนั้น `fork()`, `wait()`, `pipe()` และคำสั่ง `ps` ทำงานได้ตามปกติ อย่างไรก็ตามตัวเลขขีดจำกัดจำนวน process ในข้อ 3 อาจต่างจาก Linux ที่ติดตั้งแบบ native หรือแบบ VM เพราะ WSL2 จัดสรรหน่วยความจำให้ VM แบบไดนามิก

สรุปความคืบหน้าของรายงานฉบับนี้

ข้อ	หัวข้อ	โค้ด	ผลการทดลอง
1	fork() แบบมีและไม่มี wait()	เสร็จ	เสร็จ
2	Process status และ sleep() (zombie / orphan)	เสร็จ	เสร็จ
3	จำนวน process สูงสุดที่ fork() ได้	เสร็จ	เสร็จ
4	การสื่อสารผ่าน pipe()	ยังไม่ได้ทำ	ยังไม่ได้ทำ
5	กรณีพิเศษของ pipe (5.1–5.3)	ยังไม่ได้ทำ	ยังไม่ได้ทำ

หมายเหตุ ส่วนที่ยังไม่ได้ทำถูกทำเครื่องหมายไว้ด้วยข้อความสีแดง [ยังไม่ได้ทำ: ...] ในเนื้อรายงาน เพื่อให้หาจุดที่ต้องเติมได้ง่าย ก่อนส่งจริงต้องเติมให้ครบทุกจุดและลบคำสั่ง \todo ออก

1 fork() แบบมีและไม่มี wait()

1.1 Source code

1.1.1 1.1 โปรแกรม helloworld (ไม่มี wait())

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main(void)
6 {
7     printf("Hello World!\n");
8
9     pid_t pid = fork();
10    if (pid < 0) {
11        perror("fork");
12        exit(EXIT_FAILURE);
13    }
14
15    if (pid == 0) {
16        /* child */
17        printf("I am the child, my PID = %d, my parent PID = %d\n",
18              getpid(), getppid());
19    } else {
20        /* parent */
21        printf("I am the parent, my PID = %d, my child PID = %d\n",
22              getpid(), pid);
23    }
24
25    printf("Common line printed by PID %d\n", getpid());
26    return 0;
27 }
```

Code 1: q1_1_helloworld.c – fork() without wait()

คำอธิบายการทำงาน: เรียก fork() เพื่อสร้าง child process หากเป็น child (pid == 0) จะพิมพ์ PID ของตนเองและ parent ส่วน parent จะพิมพ์ PID ของตนเองและลูก เนื่องจากไม่มี wait() parent จึงจบการทำงานโดยไม่รอ child

1.1.2 1.2 โปรแกรมที่เพิ่ม wait()

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 int main(void)
7 {
8     printf("Hello World!\n");
9
10    pid_t pid = fork();
11    if (pid < 0) {
12        perror("fork");
13        exit(EXIT_FAILURE);
14    }
15
16    if (pid == 0) {
17        /* child */
18        printf("I am the child, my PID = %d, my parent PID = %d\n",
19              getpid(), getppid());
20    } else {
21        /* parent */
22        wait(NULL);
23        printf("I am the parent, my PID = %d, my child PID = %d\n",
24              getpid(), pid);
25    }
26
27    printf("Common line printed by PID %d\n", getpid());
28    return 0;
29 }
```

Code 2: q1_2_wait.c – fork() with wait()

คำอธิบายการทำงาน: เพิ่มการเรียก wait(NULL) ในฝั่ง parent เพื่อสั่งให้ parent หยุดรอ (Blocked) จนกว่า child จะทำงานเสร็จและคืนสถานะเรียบร้อย บังคับให้ลำดับการแสดงผลของ child เกิดขึ้นก่อน parent เสมอ

1.2 ผลการทดลอง

```
$ gcc -Wall -Wextra -o q1_1 q1_1_helloworld.c
$ ./q1_1
Hello World!
I am the parent, my PID = 3642, my child PID = 3643
Common line printed by PID 3642
I am the child, my PID = 3643, my parent PID = 3642
Common line printed by PID 3643

$ ./q1_1
Hello World!
I am the parent, my PID = 3644, my child PID = 3645
Common line printed by PID 3644
I am the child, my PID = 3645, my parent PID = 3632    <-- PPID changed (Orphan)!
Common line printed by PID 3645
```

Code 3: ผลลัพธ์ของข้อ 1.1 (ไม่มี wait())

```
$ gcc -Wall -Wextra -o q1_2 q1_2_wait.c
$ ./q1_2
Hello World!
I am the child, my PID = 3649, my parent PID = 3648
Common line printed by PID 3649
I am the parent, my PID = 3648, my child PID = 3649
Common line printed by PID 3648
```

Code 4: ผลลัพธ์ของข้อ 1.2 (มี wait())

1.3 อภิปรายผล

จากการทดลองเปรียบเทียบระหว่างโปรแกรมข้อ 1.1 (ไม่มี wait()) และข้อ 1.2 (มี wait()) สรุปความแตกต่างที่สำคัญได้ดังนี้

- **การชิงโครโนซ์และลำดับ Output:** ในข้อ 1.1 ทั้ง Parent และ Child แย่งกันทำงานอย่างอิสระ ลำดับการพิมพ์ข้อความจึงขึ้นอยู่กับ Kernel Scheduler และไม่แน่นอน (Non-deterministic) ในขณะที่ข้อ 1.2 การเรียก wait(NULL) จะทำให้ Parent หยุดรอ (Blocked) จนกว่า Child จะทำงานเสร็จ ลำดับ Output จึงมีความแน่นอน (Deterministic) โดยข้อความของ Child จะปรากฏก่อนเสมอ
- **ปรากฏการณ์ Orphan Process:** ในข้อ 1.1 มีโอกาสที่ Parent จะทำงานเสร็จและจบโปรแกรมไปก่อน ทำให้ Child กลายเป็น Orphan Process และถูกเคอร์เนลยกให้ init (หรือ Subreaper) เป็น Parent ใหม่แทน (สังเกตได้จาก PPID ของ Child ที่เปลี่ยนไปในการรันครั้งที่ 2) ส่วนในข้อ 1.2 จะไม่เกิดเหตุการณ์นี้เนื่องจาก Parent รอ Child อยู่เสมอ

2 Process Status และ sleep()

การทดลองนี้ใช้ `sleep()` เป็นเครื่องมือควบคุม ลำดับการตาย ของ parent และ child เพื่อสร้างสถานการณ์สองแบบที่ตรงข้ามกัน แล้วใช้คำสั่ง `ps` ตรวจสอบสถานะของ process ในช่วงเวลาที่สนใจ ทั้งสองโปรแกรม **จงใจไม่เรียก** `wait()` เพื่อให้เห็นพฤติกรรมที่เกิดขึ้นเมื่อ parent ไม่เก็บกวาดลูกของตนเอง

2.1 Source code

2.1.1 2.1 Child ตายก่อน โดย Parent ยังทำงานอยู่

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4
5  #define PARENT_LIFETIME 30  /* seconds the parent stays alive after child dies */
6
7  int main(void)
8  {
9      pid_t pid = fork();
10
11     if (pid < 0) {
12         perror("fork");
13         exit(EXIT_FAILURE);
14     }
15
16     if (pid == 0) {
17         /* CHILD: dies almost immediately */
18         printf("[child ] PID = %d, PPID = %d -- exiting right now\n",
19             getpid(), getppid());
20         fflush(stdout);
21         exit(0);
22     }
23
24     /* PARENT: outlives the child but never reaps it */
25     printf("[parent] PID = %d, child PID = %d\n", getpid(), pid);
26     printf("[parent] NOT calling wait(). Sleeping %d s.\n", PARENT_LIFETIME);
27     printf("[parent] Inspect now with:  ps -ef | grep -e q2_1 -e defunct\n");
28     fflush(stdout);
29
30     sleep(PARENT_LIFETIME);
31
32     printf("[parent] PID = %d exiting; the zombie child %d is now re-parented "
33         "to init/systemd and reaped.\n", getpid(), pid);
34     return 0;
35 }

```

Code 5: q2_1_child_first.c – child dies first, parent never reaps

คำอธิบายการทำงาน: หลัง `fork()` ฝั่ง child จะพิมพ์ PID ของตนเองแล้วเรียก `exit(0)` ทันที ส่วน parent เข้าสู่ `sleep(30)` ทำให้เกิดช่วงเวลา 30 วินาทีที่ child ตายไปแล้วแต่ parent ยังมีชีวิตอยู่และยังไม่ได้เรียก `wait()` จึงเป็นช่วงที่เปิดอีก terminal หนึ่งเข้าไปตรวจด้วย `ps` ได้ทัน การเรียก `fflush(stdout)` ก่อน `exit()` และก่อน `sleep()` เป็นการบังคับให้ข้อความถูกเขียนออกทันที ไม่ค้างอยู่ใน buffer

2.1.2 2.2 Parent ตายก่อน โดย Child ยังทำงานอยู่

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4
5  #define CHILD_LIFETIME 30
6
7  int main(void)
8  {
9      pid_t pid = fork();
10
11     if (pid < 0) {
12         perror("fork");
13         exit(EXIT_FAILURE);
14     }
15
16     if (pid == 0) {
17         /* CHILD: outlives the parent */
18         printf("[child ] PID = %d, PPID = %d (original parent, still alive)\n",
19             getpid(), getppid());
20         fflush(stdout);
21
22         sleep(2); /* give the parent time to die */
23
24         printf("[child ] PID = %d, PPID = %d (parent is gone -> re-parented)\n",
25             getpid(), getppid());
26         printf("[child ] Inspect now with: ps -ef | grep q2_2\n");
27         fflush(stdout);
28
29         sleep(CHILD_LIFETIME);
30
31         printf("[child ] PID = %d exiting, final PPID = %d\n",
32             getpid(), getppid());
33         exit(0);
34     }
35
36     /* PARENT: exits immediately, without waiting */
37     printf("[parent] PID = %d, child PID = %d -- exiting immediately\n",
38         getpid(), pid);
39     fflush(stdout);
40     return 0;
41 }

```

Code 6: q2_2_parent_first.c – parent dies first, child becomes an orphan

คำอธิบายการทำงาน: สลับบทบาทจากข้อ 2.1 คือ parent พิมพ์ข้อความแล้วจบการทำงานทันที ส่วน child อยู่ต่ออีก 30 วินาที จุดสำคัญคือ child เรียก `getppid()` สองครั้ง โดยมี `sleep(2)` คั่นกลาง ครั้งแรกพิมพ์ตอน parent ยังมีชีวิต ครั้งที่สองพิมพ์หลัง parent ตายไปแล้ว ทำให้เห็นค่า PPID ที่เปลี่ยนไปได้จากตัวโปรแกรมเองโดยไม่ต้องพึ่ง `ps`

2.2 ผลการทดลอง

2.2.1 2.1 Child ตายแล้ว แต่ Parent ยังทำงานอยู่

```
$ gcc -Wall -Wextra -o q2_1 q2_1_child_first.c
$ ./q2_1 &
[parent] PID = 6908, child PID = 6910
[parent] NOT calling wait(). Sleeping 30 s.
[child ] PID = 6910, PPID = 6908 -- exiting right now

$ ps -ef | grep -e q2_1 -e defunct | grep -v grep
UID      PID    PPID  C  STIME TTY      TIME    CMD
poomipat 6908    6900  0 18:58 pts/2    00:00:00 ./q2_1
poomipat 6910    6908  0 18:58 pts/2    00:00:00 [q2_1] <defunct>

$ ps -eo pid,ppid,stat,comm | grep q2_1 | grep -v grep
    PID    PPID STAT  COMMAND
    6908    6900 S+   q2_1      <- parent sleeping
    6910    6908 Z+   q2_1      <- child is a ZOMBIE

$ ps -el | grep q2_1
F S    UID      PID    PPID  C  PRI  NI ADDR SZ  WCHAN  TTY      TIME CMD
0 S   1000    6908    6900  0   80   0  -   670 hrtime pts/2    00:00:00 q2_1
1 Z   1000    6910    6908  0   80   0  -     0 -      pts/2    00:00:00 q2_1
      ^^ SZ = 0 : all memory already released
```

Code 7: ผลลัพธ์และสถานะจาก ps ในกรณี 2.1

Table 1: สรุปค่าที่สังเกตได้ในกรณี 2.1

รายการ	ค่าที่สังเกตได้
PID ของ parent	6908
PID ของ child	6910
PPID ของ child	6908 (ยังเป็น parent ตัวจริง ไม่ถูก reparent)
สถานะของ child ใน ps	Z+ แสดงผลเป็น [q2_1] <defunct>
หน่วยความจำของ child (SZ)	0

เมื่อรอจนครบ 30 วินาทีและ parent จบการทำงาน ตรวจสอบด้วย ps อีกครั้งพบว่า ไม่เหลือ process ชื่อ q2_1 อยู่ในระบบเลย แสดงว่า zombie ถูกเก็บกวาดทันที ที่ parent ตาย

2.2.2 2.2 Parent ตายแล้ว แต่ Child ยังทำงานอยู่

```
$ gcc -Wall -Wextra -o q2_2 q2_2_parent_first.c
$ ./q2_2 &
[parent] PID = 6857, child PID = 6859 -- exiting immediately
[child ] PID = 6859, PPID = 6857 (original parent, still alive)
[child ] PID = 6859, PPID = 6846 (parent is gone -> re-parented)

$ ps -ef | grep q2_2 | grep -v grep
UID      PID    PPID  C STIME TTY      TIME    CMD
poomipat 6859    6846  0 18:57 pts/2    00:00:00 ./q2_2

$ ps -eo pid,ppid,stat,comm | grep q2_2 | grep -v grep
  PID    PPID STAT  COMMAND
 6859    6846 S+   q2_2      <- normal sleeping state, NO zombie at all
```

Code 8: ผลลัพธ์และสถานะจาก ps ในกรณี 2.2

Table 2: สรุปค่าที่สังเกตได้ในกรณีที่ 2.2

รายการ	ค่าที่สังเกตได้
PID ของ parent	6857
PID ของ child	6859
PPID ของ child (ก่อน parent ตาย)	6857 (parent ตัวจริง)
PPID ของ child (หลัง parent ตาย)	6846 (ถูก reparent)
สถานะของ child ใน ps	S+ (ปกติ ไม่ใช่ zombie)

ตรวจสอบเพิ่มเติมว่า PID 6846 คือ process ไດ

```
$ ps -p 6846 -o pid=,ppid=,comm=,args=
 6846    6844 Relay(6848) /init

$ # walk the ancestry chain up to PID 1
  PID    PPID  COMMAND
 6846    6844 Relay(6848)      <- WSL /init relay : the adopting subreaper
 6844      2 SessionLeader
  2       1 init-systemd(Ub
  1       0 systemd      <- the real PID 1
```

Code 9: สายบรรพบุรุษของ process ที่รับ orphan ไปดูแล

ผลลัพธ์นี้แสดงว่า child **ไม่ได้** ถูกยกให้ PID 1 โดยตรงตามที่มักเข้าใจกัน แต่ถูกยกให้กระบวนการ /init ของ WSL ซึ่งลงทะเบียนตัวเองเป็น *child subreaper* ไปด้วย `prctl(PR_SET_CHILD_SUBREAPER)` เพราะกฎของเคอร์เนลคือยก orphan ให้ **subreaper ที่ใกล้ที่สุด** ไม่ใช่ยกให้ PID 1 เสมอไป บนระบบ Linux ที่ติดตั้งแบบ native หรือใน VM ทั่วไป คำนี้นักออกมาเป็น 1 หรือเป็น PID ของ `systemd --user`

2.3 อภิปรายผล

จากการทดลองควบคุมลำดับการจบการทำงานของกระบวนการด้วย `sleep()` ในข้อ 2.1 และ 2.2 สรุปข้อแตกต่างและประเด็นสำคัญได้ดังนี้

- **ข้อ 2.1 ปรากฏการณ์ Zombie Process:** เมื่อ Child ตายก่อนแต่ Parent ยังทำงานอยู่โดยไม่ได้เรียก `wait()` เคอร์เนลจะคืนหน่วยความจำของ Child ทันที (`SZ = 0`) แต่ยังคงจอง PID และ Process Table Entry ค้างไว้ ทำให้เกิดสถานะ `Z` หรือ `<defunct>` จนกว่า Parent จะจบโปรแกรมหรือเรียก `wait()` เพื่ออ่าน Exit Status
- **ข้อ 2.2 ปรากฏการณ์ Orphan Process และการ Reparent:** เมื่อ Parent ตายก่อนโดยที่ Child ยังคงทำงานอยู่ Child จะกลายเป็นกระบวนการกำพร้า (Orphan) เคอร์เนลจะยก Child ให้กับ Sub-reaper ที่ใกล้ที่สุด (เช่น `systemd` หรือ `/init` ของ WSL) เป็น Parent คนใหม่ทันที สังเกตได้จากค่า `PPID` ของ Child ที่เปลี่ยนไปในการเรียก `getppid()` ครั้งที่สอง
- **ความแตกต่างสำคัญและปรากฏการณ์ `<defunct>`:** Zombie (`<defunct>`) คือกระบวนการที่สิ้นสุดการทำงานแล้วแต่ยังถูกจองค้างในตารางกระบวนการ หากสะสมมากจะสิ้นเปลือง PID และทำให้สร้าง Process ใหม่ไม่ได้ ต่างจาก Orphan ที่เป็นกระบวนการซึ่งยังคงทำงานตามปกติ และเมื่อ Orphan สิ้นสุดการทำงาน Parent คนใหม่ (Subreaper) จะทำหน้าที่เรียก `wait()` เก็บกวาดให้อัตโนมัติโดยไม่เกิด Zombie ทิ้งไว้

3 จำนวน process สูงสุดที่ fork ได้

การทดลองนี้ต้องการหาว่าระบบยอมให้เรียก `fork()` ได้สำเร็จกี่ครั้ง โดยใช้โครงสร้างแบบ โซ่ (chain) ตามที่ Hint ของโจทย์แนะนำ คือทุก process สร้างลูกเพียงตัวเดียวแล้วรอด้วย `waitpid()` โซ่จึงยาวขึ้นทีละหนึ่งและคลายตัว กลับจากปลายสุดอย่างเป็นระเบียบ ซึ่ง **ไม่ใช่** fork bomb และหยุดได้ทุกเมื่อ

3.1 Source code

3.1.1 ส่วนหลัก: ฟังก์ชัน `chain()` และการรายงานผล

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <errno.h>
5  #include <unistd.h>
6  #include <sys/wait.h>
7  #include <sys/resource.h>
8
9  #define HARD_CAP 200000      /* never build a chain longer than this */
10
11 struct result {
12     int count;                /* number of successful fork() calls */
13     int err;                  /* errno that stopped us (0 = hit HARD_CAP) */
14 };
15
16 static int pfd[2];
17
18 /* Send the tally up the chain to the root and leave. */
19 static _Noreturn void report(int count, int err)
20 {
21     struct result r;
22     r.count = count;
23     r.err = err;
24     if (write(pfd[1], &r, sizeof r) != (ssize_t)sizeof r)
25         _exit(1);
26     _exit(0);
27 }
28
29 #pragma GCC diagnostic push
30 #pragma GCC diagnostic ignored "-Winfinite-recursion"
31
32 /* I am the process produced by fork() call number `level`. */
33 static void chain(int level)
34 {
35     pid_t pid;
36
37     if (level >= HARD_CAP)
38         report(level, 0);      /* stopped by our own ceiling */
39
40     pid = fork();
41
42     if (pid < 0)
43         /* fork #(level+1) failed -> `level` forks succeeded in total */
44         report(level, errno);
45

```

```
46     if (pid == 0)
47         chain(level + 1);      /* the child keeps digging */
48
49     /* parent of this link: hold still until the chain below me unwinds */
50     waitpid(pid, NULL, 0);
51     _exit(0);
52 }
53
54 #pragma GCC diagnostic pop
```

Code 10: q3_forkcount.c – the chain and the reporting path

คำอธิบายการทำงาน: `chain(level)` หมายถึง “ฉันคือ process ที่เกิดจาก `fork()` ครั้งที่ `level`” แต่ละตัวจะ `fork()` หนึ่งครั้ง ถ้าสำเร็จ ลูกจะเรียก `chain(level+1)` ชุติลต่อไป ส่วนพ่อจะติดอยู่ที่ `waitpid()` ไม่ทำอะไรจนกว่าโซ่ข้างล่างจะคลายกลับมา ทำให้มีเพียง process เดียวเท่านั้นที่ทำงานอยู่ในแต่ละขณะ เมื่อ `fork()` ล้มเหลว process ที่ลึกที่สุดจะส่งผลรวมและ `errno` กลับไปให้ root ผ่าน pipe ด้วย `report()` ส่วน `HARD_CAP` เป็นเพดานของโปรแกรมเองไว้กันพลาด และคำสั่ง `#pragma` จำเป็นต้องมีเพราะ gcc มองว่า `chain()` เป็น infinite recursion ซึ่งเป็นการเตือนที่ผิดพลาด เนื่องจากการเรียกซ้ำเกิดใน process ใหม่คนละตัวและหยุดเมื่อ `fork()` ล้มเหลว

3.1.2 ส่วน main(): กำหนดเพดานและรับผลลัพธ์

```

1  int main(int argc, char *argv[])
2  {
3      pid_t pid;
4      struct result r = { 0, 0 };
5      struct rlimit rl;
6
7      if (argc > 1) {          /* optional artificial ceiling for rehearsal */
8          rlim_t want = (rlim_t)strtoul(argv[1], NULL, 10);
9          if (getrlimit(RLIMIT_NPROC, &rl) == 0) {
10             rl.rlim_cur = want;
11             if (rl.rlim_max != RLIM_INFINITY && rl.rlim_cur > rl.rlim_max)
12                 rl.rlim_cur = rl.rlim_max;
13             if (setrlimit(RLIMIT_NPROC, &rl) != 0)
14                 perror("setrlimit");
15         }
16     }
17
18     if (getrlimit(RLIMIT_NPROC, &rl) == 0)
19         printf("RLIMIT_NPROC in effect: soft = %ld, hard = %ld\n",
20             (long)rl.rlim_cur, (long)rl.rlim_max);
21
22     printf("Root PID = %d. Forking a chain until fork() fails ...\n", getpid());
23     fflush(stdout);
24
25     if (pipe(pfd) < 0) {
26         perror("pipe");
27         exit(EXIT_FAILURE);
28     }
29
30     pid = fork();
31     if (pid < 0) {
32         r.count = 0;
33         r.err = errno;
34     } else if (pid == 0) {
35         close(pfd[0]);
36         chain(1);          /* this process came from fork() #1 */
37         _exit(0);          /* unreachable */
38     } else {
39         close(pfd[1]);      /* only the descendants write */
40         waitpid(pid, NULL, 0);
41         if (read(pfd[0], &r, sizeof r) != (ssize_t)sizeof r) {
42             fprintf(stderr, "could not read result from pipe\n");
43             exit(EXIT_FAILURE);
44         }
45         close(pfd[0]);
46     }
47
48     printf("fork() succeeded %d times.\n", r.count);
49     if (r.err == 0)
50         printf("Stopped by the program's own HARD_CAP (%d), not by the OS.\n",
51             HARD_CAP);
52     else
53         printf("fork() #%d failed with errno %d (%s).\n",
54             r.count + 1, r.err, strerror(r.err));
55
56     return 0;
57 }

```

Code 11: q3_forkcount.c – main(): optional ceiling and result collection

คำอธิบายการทำงาน: `main()` สร้าง pipe ก่อนแล้วจึง `fork()` เพื่อเริ่มไช่ ฝั่ง root ปิดปลายเขียน (`pfid[1]`) เพราะมีแต่ลูกหลานที่จะเขียน ส่วนลูกตัวแรกปิดปลายอ่าน (`pfid[0]`) การที่ root เรียก `waitpid()` ก่อนแล้วค่อย `read()` ไม่ทำให้เกิด deadlock เพราะข้อมูลขนาด 8 ไบต์ถูกเขียนค้างไว้ใน buffer ของ pipe ตั้งแต่ตอนที่ process ปลายสุดยังมีชีวิตอยู่ และเนื่องจากขนาดน้อยกว่า `PIPE_BUF` การเขียนจึงเป็น atomic ไม่มีทางถูกแทรก ส่วนอาร์กิวเมนต์ `argv[1]` เป็นทางเลือกให้ตั้งเพดานจำลองด้วย `setrlimit()` เพื่อซ้อมการทดลองในระดับที่ปลอดภัยก่อนรันจริง

3.2 ผลการทดลอง

ทำการทดลอง 2 ชุดด้วยเพดานต่างกัน เพื่อยืนยันว่าข้อสรุปไม่ได้ขึ้นกับขนาดของเพดาน ชุด A ใช้ `ulimit -u 300` ซึ่งรันเร็ว ส่วนชุด B ใช้ `ulimit -u 6000` ซึ่งใกล้เคียงสถานการณ์จริงมากกว่า

3.2.1 3.1 การวัดครั้งแรก

```
$ gcc -Wall -Wextra -o q3_forkcount q3_forkcount.c

--- Set A ---
$ ulimit -u 300
$ ./q3_forkcount
RLIMIT_NPROC in effect: soft = 300, hard = 300
Root PID = 25252. Forking a chain until fork() fails ...
fork() succeeded 255 times.
fork() #256 failed with errno 11 (Resource temporarily unavailable).

--- Set B ---
$ ulimit -u 6000
$ ./q3_forkcount
RLIMIT_NPROC in effect: soft = 6000, hard = 6000
Root PID = 6921. Forking a chain until fork() fails ...
fork() succeeded 5953 times.
fork() #5954 failed with errno 11 (Resource temporarily unavailable).
```

Code 12: ผลลัพธ์การวัดครั้งแรกของทั้งสองชุด

3.2.2 3.2 เมื่อมีโปรแกรมอื่นทำงานค้างอยู่

ชุด B ทำการวัดต่อเนื่อง 3 เฟสภายใต้เงื่อนไขเดียวกัน โดยตรวจนับจำนวน thread ของผู้ใช้ก่อนวัดทุกครั้ง เพื่อยืนยันว่าสภาพระบบกลับสู่จุดเริ่มต้นจริง

```
===== PHASE 1 : before starting any extra program =====
UID threads now: 8
$ ./q3_forkcount
fork() succeeded 5953 times.
fork() #5954 failed with errno 11 (Resource temporarily unavailable).

===== PHASE 2 : with 100 extra programs running =====
$ for i in $(seq 1 100); do sleep 900 & done
UID threads now: 108
$ ./q3_forkcount
fork() succeeded 5853 times.                <- exactly 100 fewer
fork() #5854 failed with errno 11 (Resource temporarily unavailable).

===== PHASE 3 : after stopping all extra programs =====
$ kill $PIDS
UID threads now: 8
$ ./q3_forkcount
fork() succeeded 5953 times.                <- back to the original value
fork() #5954 failed with errno 11 (Resource temporarily unavailable).
```

Code 13: ผลลัพธ์ทั้งสามเฟสของชุด B (ulimit -u 6000)

Table 3: เปรียบเทียบจำนวน `fork()` ที่สำเร็จในแต่ละสถานการณ์

ชุด	สถานการณ์	process อื่นที่เพิ่ม	<code>fork()</code> สำเร็จ (ครั้ง)
A (เพดาน 300)	ก่อนรันโปรแกรมเพิ่ม	–	255
A (เพดาน 300)	ระหว่างรันโปรแกรมเพิ่ม	20	235
A (เพดาน 300)	หลังปิดโปรแกรมทั้งหมด	–	256
B (เพดาน 6000)	ก่อนรันโปรแกรมเพิ่ม	–	5953
B (เพดาน 6000)	ระหว่างรันโปรแกรมเพิ่ม	100	5853
B (เพดาน 6000)	หลังปิดโปรแกรมทั้งหมด	–	5953

3.3 อภิปรายผล

จากการทดลองวัดจำนวน `fork()` ที่สำเร็จภายใต้สภาพระบบที่ต่างกัน สรุปประเด็นสำคัญได้ดังนี้

- **ขีดจำกัดที่แท้จริงคือ `RLIMIT_NPROC`:** `fork()` return `-1` พร้อม `errno = 11` ซึ่งคือ `EAGAIN` (Resource temporarily unavailable) เหมือนกันทุกครั้งในทุกการทดลอง ไม่ใช่ `ENOMEM` แสดงว่าสิ่งที่หยุดเราคือโควตาจำนวน process ต่อผู้ใช้ ไม่ใช่หน่วยความจำหรือขนาดของตารางกระบวนการ
- **จำนวนที่ได้ไม่เท่ากับเพดานเพราะโควตาถูกใช้ไปก่อนแล้ว:** เพดาน 300 ได้ 255 ครั้ง (ส่วนต่าง 45) และเพดาน 6000 ได้ 5953 ครั้ง (ส่วนต่าง 47) เนื่องจาก `RLIMIT_NPROC` นับ process ของ UID นั้น ทั้ง user namespace ไม่ใช่ namespace ที่โปรแกรมเราสร้าง ส่วนต่างจึงคือ process ที่ผู้ใช้มีอยู่ก่อนแล้ว เช่น shell, WSL relay และ session อื่น ที่เปิดค้างไว้ สังเกตว่า `ps` มองเห็นเพียง 8 ตัวใน session ปัจจุบัน แต่ถูกหักไป 47 แสดงว่ามีอีกราว 39 ตัวที่อยู่นอก session ปัจจุบัน แต่ยังถูกนับรวมในโควตาเดียวกัน
- **จำนวนลดลงตามจำนวนโปรแกรมที่รันค้างแบบหนึ่งต่อหนึ่ง:** เมื่อเปิดโปรแกรม `sleep` ค้างไว้ 100 ตัว จำนวน `fork()` ที่สำเร็จลดลงจาก 5953 เหลือ 5853 คือลดลง **100 พอดีเป๊ะ** ไม่ขาดไม่เกิน เช่นเดียวกับชุด A ที่เปิด 20 ตัวแล้วลดลง 20 พอดี ยืนยันว่าโควตานี้เป็นทรัพยากรที่แชร์กันทั้งผู้ใช้ ไม่ใช่โควตาต่อโปรแกรม
- **ค่าจะกลับมาเท่าเดิมก็ต่อเมื่อควบคุมตัวแปรได้ครบ:** ชุด B กลับมาที่ 5953 เท่าเดิมเป๊ะ เพราะปิดเฉพาะ PID ที่เปิดเองและตรวจนับ thread ยืนยันว่ากลับมาที่ 8 เท่าเดิมก่อนวัดซ้ำ ในขณะที่ชุด A ได้ 256 ครั้ง ซึ่งมากกว่าเดิม 1 เพราะระหว่างการทดลองมี process อื่นของผู้ใช้เข้าไปพอดี ทำให้โควตาว่างเพิ่มขึ้น จึงสรุปได้ว่าตัวเลขที่วัดได้เป็นค่าที่ขึ้นกับสภาพ ของระบบ ณ ขณะนั้น ไม่ใช่ค่าคงที่ประจำเครื่อง
- **ต้นทุนของโครงสร้างแบบ recursive chain:** การวัดแต่ละครั้งในชุด B ใช้เวลา 755–895 วินาที และใช้หน่วยความจำสูงสุด ราว 13 GB ที่ความยาวโซ่ประมาณ 5,900 process เพราะ process ที่ระดับ N มี stack frame สะสม N เฟรม ทุกครั้งที่ `fork()` เคอร์เนลต้องคัดลอก page table ที่ใหญ่ขึ้นเรื่อย ๆ ทำให้ทั้งเวลาและหน่วยความจำโตแบบ $O(N^2)$ นี่คือเหตุผลที่ไม่สามารถรันจนถึงเพดานจริง 62,827 ได้บนเครื่องที่มี หน่วยความจำ 15.7 GB และเป็นเหตุผลที่ต้องกำหนด `ulimit -u` ให้ต่ำลงก่อนการทดลองทุกครั้ง

4 การสื่อสารผ่าน Pipe

[ยังไม่ได้ทำ: ยังไม่ได้เขียนโปรแกรมข้อ 4 – ต้องสร้างไฟล์ q4_pipe.c]

ผลลัพธ์ที่โจทย์กำหนด ต้องได้ output ตามรูปแบบนี้ โดย child เป็นผู้ส่ง และ parent เป็นผู้รับ

```
Child: Child PID: xxxxxx
Parent: Parent PID: yyyyyy
Parent: Hello from child PID: xxxxxx
```

4.1 Source code

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <string.h>
4 #include <sys/wait.h>
5
6 int main(void)
7 {
8     int fd[2];
9     /* TODO: pipe(fd) -> fork()
10      *      child : close(fd[0]) then write()
11      *      parent: close(fd[1]) then read()
12      */
13     return 0;
14 }
```

Code 14: q4.c – child เป็นผู้ส่ง parent เป็นผู้รับ

คำอธิบายการทำงาน [ยังไม่ได้ทำ: อธิบายลำดับ pipe() → fork() → ปิดปลายที่ไม่ใช้ → write() / read() และเหตุผลที่ต้องปิดปลายที่ไม่ใช้ เพราะถ้าไม่ปิดฝั่ง write ให้ครบทุก process แล้ว read() จะไม่มีวันได้รับ EOF และจะ block ค้างตลอดไป]

4.2 ผลการทดลอง

```
$ ./q4
```

Code 15: ผลลัพธ์ของข้อ 4

4.3 อภิปรายผล

[ยังไม่ได้ทำ: เขียนอภิปราย] ประเด็นที่ควรกล่าวถึง เช่น ทำไม pipe จึงเป็นการสื่อสารทางเดียว (uni-directional) ทำไม file descriptor จึงถูกสืบทอดไปยัง child ได้ และ read() มีพฤติกรรม blocking อย่างไร

5 การทดลองกรณีพิเศษของ Pipe

[ยังไม่ได้ทำ: ยังไม่ได้ทำการทดลองข้อ 5 ทั้งหมด – ต้องดัดแปลงโค้ดข้อ 4 เป็นสามเวอร์ชัน]

5.1 5.1 ผู้ส่งพยายามอ่านข้อความของตัวเองกลับ

วิธีทดลอง [ยังไม่ได้ทำ: ให้ child ไม่ปิด fd[0] แล้วลอง read() กลับหลังจาก write() เสร็จ]

```
1 /* TODO */
```

Code 16: ส่วนของโค้ดที่แก้ไขในข้อ 5.1

Code 17: ผลลัพธ์ของข้อ 5.1

สิ่งที่สังเกตได้และการอภิปราย [ยังไม่ได้ทำ: อธิบายว่าสุดท้ายใครได้ข้อมูลไป เพราะ pipe เป็นคิวแบบ FIFO ที่ข้อมูลถูกอ่านแล้วหมดไป ไม่ใช่การ broadcast ผู้อ่านคนแรกที่มาถึงจะดึงข้อมูลออกไปอีกฝ่ายจึงไม่เหลืออะไรให้อ่านและอาจ block ต่าง]

5.2 5.2 ผู้รับอ่านก่อนที่ผู้ส่งจะส่ง

วิธีทดลอง [ยังไม่ได้ทำ: ให้ child sleep() สักครู่ก่อน write() เพื่อบังคับให้ parent เรียก read() ก่อนที่จะมีข้อมูลใน pipe]

```
1 /* TODO */
```

Code 18: ส่วนของโค้ดที่แก้ไขในข้อ 5.2

Code 19: ผลลัพธ์ของข้อ 5.2

สิ่งที่สังเกตได้และการอภิปราย [ยังไม่ได้ทำ: อธิบายพฤติกรรม blocking read ว่า read() จะรอจนกว่าจะมีข้อมูล และเงื่อนไขที่ read() จะ return 0 (EOF) คือเมื่อ write-end ถูกปิดหมดทุก process]

5.3 5.3 ผู้ส่งส่งหลายข้อความก่อนที่ผู้รับจะอ่าน

วิธีทดลอง [ยังไม่ได้ทำ: ให้ child write() หลายครั้งติดกัน แล้ว parent ค่อยเรียก read() ทีหลัง]

```
1 /* TODO */
```

Code 20: ส่วนของโค้ดที่แก้ไขในข้อ 5.3

Code 21: ผลลัพธ์ของข้อ 5.3

สิ่งที่สังเกตได้และการอภิปราย [ยังไม่ได้ทำ: อธิบายว่า pipe เป็น byte stream ที่ไม่รักษาขอบเขตของข้อความ (no message boundary) ข้อความหลายชิ้นจึงถูกอ่านรวมกันมา ใน read() ครั้งเดียวได้ และกล่าวถึงขนาด buffer ของ pipe (ปกติ 64 KB บน Linux) ที่ทำให้ write() block เมื่อ buffer เต็ม]

5.4 สรุปผลการทดลองข้อ 5

Table 4: สรุปพฤติกรรมของ pipe ในแต่ละสถานการณ์

สถานการณ์	ผลที่สังเกตได้	คำอธิบาย
5.1 ผู้ส่งอ่านข้อความตัวเอง		
5.2 ผู้รับอ่านก่อนส่ง		
5.3 ส่งหลายข้อความ ก่อนอ่าน		

สรุปผลการทดลอง

สิ่งที่ได้เรียนรู้จากข้อ 1

1. `fork()` คือการเรียกฟังก์ชันเพียงครั้งเดียวที่ return สองครั้ง เป็นกลไกพื้นฐานที่ทำให้โค้ดชุดเดียวแตกออกเป็นสองเส้นทางการทำงาน
2. ถ้าไม่มีกลไกชิงโครโนส์ ลำดับการทำงานของ parent กับ child เป็นสิ่งที่ **คาดเดาไม่ได้** และขึ้นกับ scheduler ล้วน ๆ การรันโปรแกรมเดิมซ้ำสามครั้ง ให้ผลไม่ซ้ำกันเลย
3. `wait()` เปลี่ยนโปรแกรมจาก non-deterministic เป็น deterministic และเป็นวิธีป้องกัน zombie process ที่ตรงไปตรงมาที่สุด
4. ได้เห็น **orphan process** และการ reparent โดยบังเอิญจากการรันข้อ 1.1 ซึ่งเป็นปรากฏการณ์เดียวกับข้อ 2.2 ต้องการให้ทดลอง
5. `fork()` คัดลอก **ทุกอย่าง** ใน address space รวมถึง stdio buffer ที่ยังไม่ได้ flush ทำให้เกิดข้อความซ้ำเมื่อ redirect output ลงไฟล์ เป็นกับดักที่พบได้บ่อยและแก้ได้ด้วย `fflush(stdout)` ก่อน `fork()`

ปัญหาที่พบและวิธีแก้

- ข้อความ **"HeLlo WorLd!"** ปรากฏสองครั้ง เมื่อ redirect output ลงไฟล์ ตอนแรกเข้าใจผิดว่าโปรแกรมเขียนผิด ภายหลังพบว่าเกิดจาก stdio buffering แก้โดยเพิ่ม `fflush(stdout)`; ก่อนเรียก `fork()`
- ผลการรันไม่ซ้ำเดิม ทำให้บันทึกผลได้ยาก จึงต้องรันหลายครั้งและรายงานทุกครั้ง เพื่อแสดงให้เห็นความไม่แน่นอน แทนที่จะรายงานเพียงครั้งเดียว

งานที่ยังเหลือ **[ยังไม่ได้ทำ: ข้อ 2, 3, 4 และ 5 – ต้องเขียนโปรแกรม รันเก็บผล และเขียนอภิปรายให้ครบก่อนส่ง]**

เอกสารอ้างอิง

1. Linux manual pages: `fork(2)`, `wait(2)`, `pipe(2)`, `ps(1)`, `setvbuf(3)`.
2. เอกสารประกอบการสอน CPE 333 Operating Systems, บทที่ว่าด้วย Process Management.